Aprendizado de máquina - atividade discente supervisionada 2

Prof. Mozart Hasse

LEIA ATENTAMENTE TODAS AS INTRUÇÕES ATÉ O FINAL DA ÚLTIMA PÁGINA. CADA PALAVRA CONTA!

O objetivo da equipe é construir de ponta a ponta uma API de chat baseada em RAG. Para facilitar os testes pode-se fazer uma interface bem rudimentar, mas o objetivo avaliado será **responder a perguntas formuladas pelo usuário sobre os assuntos cobertos pelos PDFs, citando fontes e trechos, ou indicando que o assunto não está coberto pelo material disponível.**

Use exclusivamente os PDFs fornecidos na lista a seguir, com dados oficiais publicados pelo governo do Paraná:

https://www.ipardes.pr.gov.br/sites/ipardes/arquivos_restritos/files/documento/2023-09/desenvolvimento_paranaense.pdf

https://www.ipardes.pr.gov.br/sites/ipardes/arquivos_restritos/files/documento/2026-02/Analise_Conjuntural_julho_agosto_2025.pdf

https://www.ipardes.pr.gov.br/sites/ipardes/arquivos_restritos/files/documento/2025-12/Avaliacoes%20Politicas%20Publicas%20Brasil_revisao%20escopo.pdf

O conteúdo destes documentos deve ser indexado usando ferramentas de código aberto, SEM USO DE APIs ou serviços externos que dependam de internet. TODA A EXECUÇÃO DE TODAS AS ETAPAS DO PROCESSO DEVE SER FEITA A PARTIR DE UMA MÁQUINA LOCAL SEM ACESSO A SERVIÇOS DE LLMs e/ou de indexação na internet (uso permitido apenas para preparação de ambiente, ou seja, baixar bibliotecas python e modelos públicos de LLMs).

NÃO É PERMITIDO complementar estes dados com informações de outras bases de dados, use apenas os dados fornecidos no arquivo ou valores deduzidos através de fórmulas. Se quiser e achar conveniente, pode preprocessar os dados com as próprias LLMs usadas em sua solução, sempre com soluções de código aberto e que não dependam de nenhum serviço da internet.

Qualquer modelo de LLM usado no trabalho pode ter NO MÁXIMO 9,9 bilhões de parâmetros.

É importante salientar que, devido à imposição de usar apenas ferramentas de código aberto, o resultado gerado será bastante limitado. Porém, o objetivo é exatamente fazer com que a equipe identifique e busque soluções para as consequências do uso de LLMs nos diversos estágios do processo, com todos os erros típicos amplificados, assim como a boa implementação de soluções e ajustes para tratá-los.

A equipe precisará:

- Planejar e implementar o processo de separação de trechos dos documentos em blocos menores (chunking)
- Realizar um cuidadoso e extenso tratamento de dados para indexação
- Escolher o conteúdo a ser indexado e a forma de armazenamento e acesso
- Escolher as etapas e complementos no processo de busca e indexação
- Adequação à janela de contexto da LLM escolhida
- Prompt final gerado com uso do RAG

Critérios de avaliação

Organização e clareza do código: (20% da nota), solução seguindo as melhores práticas de mercado, comentários com a justificativa para as escolhas feitas na análise e na vetorização. Durante a execução o código deve rodar EXCLUSIVAMENTE EM UMA MÁQUINA SEM CONEXÃO COM A INTERNET.

Testes unitários (20% da nota): testes unitários automatizados e que cubram, não só a cobertura de código quanto a qualidade da vetorização/preprocessamento, usando métricas decididas e implementadas pela equipe.

Qualidade do tratamento de dados (30% da nota): preprocessamento, filtragem de dados irrelevantes, chunking e demais operações antes da vetorização.

Qualidade do RAG (30% da nota): O professor usará seu baseline com:

- perguntas que cubram um tópico de cada documento fornecido
- perguntas não relacionadas aos tópicos para verificar se o modelo gerar respostas além do seu escopo determinado, que não pode ocorrer
- Perguntas que cubram vários tópicos de vários documentos, onde se avaliará se o modelo vai completar lacunas inexistentes dos documentos com informações inventadas, o que também não pode ocorrer
- Quais techos de quais documentos foram escolhidos para cada resposta e qual o prompt final montado pela aplicação para cada pergunta com os dados recuperados via RAG. Em aplicações comerciais essa parte não é mostrada mas para este trabalho o professor EXIGE ver em separado o resultado dessa operação para fins de avaliação.

Cabe a ressalva de que, até em modelos de última geração, evitar falhas nas respostas é impossível. O que se avaliará é a correta e cuidadosa implementação de um PROCESSO coerente e bem planejado, capaz de identificar as principais falhas e tratá-las de maneira razoável, com possibilidade de fácil expansão.

Instruções para entrega

O trabalho deve ser entregue em UM arquivo ZIP contendo:

- **Código-fonte do processo de extração e tratamento de dados, incluindo a geração e armazenamento dos dados de chunking e vetorização;**
- **Os arquivos intermediários com os dados vetorizados e trechos dos documentos já tratados, prontos para uso no chat caso o processor não queira regerar todo o banco de dados a partir do código-fonte;**
- **Código-fonte do processo de geração das respostas usando RAG. A interface pode ser a mais simples possível, pois o que será avaliado é a resposta para a pergunta e os trechos usados como base. Para cada pergunta realizada pelo usuário este código deve gerar DUAS saídas:**

    - **O prompt e os trechos dos documentos usados para enriquecer a resposta**
    - **A resposta final gerada pela LLM para o usuário**
- **Caso seja necessário algum aplicativo para o banco de dados vetorial, deve-se ter instruções claras e configuração para execução local via docker ou docker-compose. Quero rodar na minha máquina digitando um comando.**
- **USE O BOM SENSO, respeite o limite de tamanho e coloque no seu ZIP o que for produzido pela equipe e não for possível baixar pronto da internet.**

Observações gerais

O trabalho pode ser feito em equipes de até 4 alunos. A EQUIPE TODA É IGUALMENTE RESPONSÁVEL PELO SUCESSO DO TRABALHO.

CUIDADO: aqui está se avaliando tanto o resultado gerado quanto o código e algoritmo escolhidos. Apresente um código compreensível por todos os membros da equipe, especialmente quanto aos parâmetros escolhidos.

Não use este documento com ferramentas de IA pois o professor vê isso como uma atitude, hum... *contraproducente*. Ao pedir algo a uma IA, explique o que você precisa com as **SUAS** palavras.

É TERMINANTEMENTE PROIBIDO compartilhar arquivos entre equipes. Qualquer tentativa de fazer isso implicará na atribuição de nota ZERO a TODOS os membros de TODAS as equipes envolvidas. Casos suspeitos passarão por prova de autoria, portanto todos os membros da equipe devem saber como o código funciona e os motivos de cada escolha feita.

Cabe lembrar que compartilhar VERBALMENTE caminhos bem e mal sucedidos é permitido. O único cuidado é não compartilhar também os **eventuais valores de parâmetros de configuração e prompts que alimentam o RAG**.

ANEXOS

Seguem algumas SUGESTÕES de ferramentas e técnicas para resolver o problema proposto. USE POR SUA CONTA E RISCO. Recomenda-se a leitura para entender as tecnologias e opções disponíveis em cada caso.

Ferramentas por Etapa — Visão Consolidada

Camada	Opções
Extração de PDF	pdfplumber · PyMuPDF (fitz) · Docling (IBM) · PDFMiner · Camelot *(tabelas)*
Pré-processamento	spaCy `pt_core_news_lg` · NLTK · ftfy · unicodedata *(stdlib)* · regex *(stdlib)*
Chunking	LangChain `RecursiveCharacterTextSplitter` · Chonkie `SemanticChunker` · LlamaIndex `SentenceSplitter` · NLTK `sent_tokenize` *(manual)*
Embeddings	`intfloat/multilingual-e5-large` · `neuralmind/bert-large-portuguese-cased` · `rufimelo/bert-large-portuguese-cased-sts` · `paraphrase-multilingual-MiniLM-L12-v2`
Vector Store	ChromaDB · Qdrant *(local/in-memory)* · FAISS · LanceDB
Re-ranking	`cross-encoder/ms-marco-MiniLM-L-6-v2` · `unicamp-dl/mt5-3b-mmarco-en-pt` · `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`
LLM local	Gemma 3:4b · Gemma 3:9b · Llama 3.1:8b · Qwen2.5:7b · Mistral 7B · Phi-4-mini *(3.8B)*
Orquestração	LangChain (LCEL) · Haystack 2.x · LlamaIndex · DSPy
API	FastAPI · Flask · LiteStar
UI de testes	Gradio · Streamlit · Panel
Runtime LLM	Ollama · llama.cpp · LM Studio *(GUI)* · vLLM *(se houver GPU)*

Etapa	Sugestões de Ferramentas / Abordagens
Extração de Texto dos PDFs	- Unstructured.io (hi_res) - PyMuPDF + pdfplumber - Marker - Nougat - camelot-py + tabula-py
Limpeza e Tratamento de Dados	- Reprocessamento de texto com LLM local - Correção de quebras de linha e artefatos - Conversão de tabelas para Markdown ou JSON - Normalização de texto
Enriquecimento de Metadados	- Extração automática de título da seção, número da página e nome do documento - Identificação de seções hierárquicas - Adição de metadados por chunk
Chunking	- RecursiveCharacterTextSplitter - MarkdownHeaderTextSplitter - SemanticChunker - SentenceSplitter - Hierarchical Node Parser - Chunking por seções + subseções - Manter tabelas inteiras quando possível
Vetorização (Embeddings)	- Modelos de embedding multilíngues open-source com até 9.9B - BGE-M3 - Qwen3-Embedding (versões ≤ 8B) - Snowflake-arctic-embed - Nomic-embed
Armazenamento Vetorial	- Chroma - Qdrant (modo local) - LanceDB - FAISS - pgvector (PostgreSQL local)
LLM para Geração	- Modelos Instruct open-source com no máximo 9.9B parâmetros - Qwen3-8B-Instruct - Tucano2-7B / 3.7B - Gemma-3-9B (ou variantes) - Llama-3.1-8B-Instruct - DeepSeek-V2-Lite (se ≤ 9.9B)
Retrieval	- Similarity Search (cosine) - MMR (Maximal Marginal Relevance) - Top-k + reranking - Metadata filtering por documento ou seção
Reranking	- Cross-encoder leve (BGE-reranker, etc.) - Reordenação dos resultados antes da geração
Construção do Prompt Final	- Prompt com contexto recuperado - Instruções claras de fidelidade aos documentos - Regra obrigatória de "não está coberto" - Formato padronizado de citação (documento, página, trecho) - Adequação ao tamanho da janela de contexto da LLM
Geração da Resposta	- Geração com LLM local - Controle de temperatura baixa - Post-processamento para garantir citações
Interface e API	- FastAPI (backend) - Gradio ou Streamlit (interface rudimentar) - Endpoint de chat com retorno de resposta + fontes citadas

Etapa	Objetivo	Opções / Técnicas	Ferramentas / Modelos
1. Ingestão de PDFs	Ler arquivos e extrair conteúdo bruto	Leitura página a página; extração de metadados; detecção de estrutura básica	pymupdf (fitz), pdfminer.six, pypdf, unstructured
2. Parsing e Estruturação	Converter PDF em texto utilizável	Separação por página, blocos, títulos; identificação de seções	pymupdf, unstructured, pdfplumber
3. Limpeza de texto	Remover ruído e inconsistências	Remoção de headers/footers; normalização de espaços; correção de quebras de linha; deduplicação	regex, nltk, spaCy
4. Filtragem de conteúdo	Eliminar partes irrelevantes	Remoção de sumários, índices, tabelas quebradas; heurísticas de densidade textual	regras heurísticas, estatísticas simples
5. Normalização linguística	Padronizar texto	lowercasing (opcional), remoção de stopwords (opcional), stemming/lemmatization	nltk, spaCy
6. Segmentação inicial	Dividir texto em unidades básicas	Por parágrafos, sentenças ou blocos semânticos	nltk, spaCy, regex
7. Chunking	Criar blocos para recuperação	Fixed-size; sliding window; paragraph-based; semantic chunking; hierarchical chunking	implementações próprias, LangChain (offline), LlamaIndex (offline)
8. Overlap entre chunks	Preservar contexto	Overlap fixo; overlap adaptativo; sem overlap	implementação própria
9. Enriquecimento de metadata	Garantir rastreabilidade	Inclusão de documento, página, seção, posição no texto	estrutura JSON customizada
10. Vetorização (embeddings)	Representar semanticamente os chunks	Sentence embeddings; embeddings multilíngues; embeddings instrucionais	sentence-transformers (MiniLM, MPNet, BGE-small/base), Instructor embeddings
11. Normalização de vetores	Preparar para busca	L2 normalization; cosine similarity preparation	numpy, faiss utilities
12. Indexação vetorial	Armazenar embeddings	Flat index; IVF; HNSW; PQ (Product Quantization)	FAISS, ChromaDB, Annoy, hnswlib

	para busca eficiente		
13. Armazenamento de metadata	Associar vetores ao conteúdo original	Banco key-value; arquivos JSON; SQLite	SQLite, JSON, TinyDB
14. Query encoding	Converter pergunta em embedding	Mesmo modelo dos documentos ou modelo distinto	sentence-transformers
15. Retrieval inicial	Buscar chunks relevantes	Top-k similarity; threshold filtering	FAISS, hnswlib, Annoy
16. Pós-processamento de resultados	Refinar candidatos	Remoção de duplicados; diversidade (MMR); filtragem por score	implementação própria, sklearn
17. Re-ranking	Melhorar ordenação semântica	Cross-encoder ranking; scoring supervisionado	cross-encoder MiniLM, BGE reranker
18. Seleção de contexto	Escolher chunks finais	Top-k final; limite por tokens; compressão de contexto	heurísticas próprias
19. Compressão de contexto (opcional)	Reduzir tokens mantendo informação	Summarization; keyword extraction; sentence selection	LLM local (<9.9B), TextRank
20. Construção do prompt	Estruturar entrada da LLM	Prompt com instruções restritivas; inclusão de fontes; delimitação clara de contexto	templates próprios
21. Geração com LLM	Produzir resposta final	Inference com temperatura baixa; controle de max tokens	llama.cpp, Ollama, vLLM (local) + modelos ≤9.9B (Llama 3 8B, Mistral 7B, Gemma 2 9B)
22. Controle de alucinação	Garantir fidelidade	Instruções explícitas; fallback “não encontrado”; threshold de similaridade	heurísticas + prompt engineering
23. Extração de citações	Justificar resposta	Highlight de trechos; referência a página/documento	regex, alinhamento de texto
24. API	Expor sistema	Endpoint REST; serialização JSON	FastAPI, Flask
25. Interface	Interação com usuário	UI simples; CLI; chat básico	Streamlit, Gradio, terminal
26. Avaliação	Medir qualidade	Precision@k; recall; avaliação manual; groundedness	scripts próprios, datasets internos

27. Logging e debugging	Diagnóstico	Log de queries, chunks retornados, scores	logging padrão Python
28. Otimização	Melhorar desempenho	Batch embeddings; cache; quantização de modelos	numpy, faiss, quantização GGUF
29. Persistência	Reuso do índice	Salvamento/carregamento de embeddings e índices	FAISS save/load, arquivos locais
30. Orquestração	Pipeline completo	Scripts encadeados; DAG simples	Makefile, bash, Python scripts